E4 Educator v2.0

T14

OTA updates

Tier 2 · Tutorial 7 of 9

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- What OTA is and how the ESP32 dual-partition OTA mechanism works
- Why the partition table must reserve OTA space and how to verify it
- How to add ElegantOTA to any existing E4 project in under 10 minutes
- How to trigger firmware updates from a browser on the local network
- How to show OTA progress on the TFT display with a progress bar
- How to version firmware correctly and display it on the dashboard
- OTA safety practices — rollback, watchdog, and update-safe firmware design
- How to update the LittleFS filesystem image over OTA
- The complete Tier 2 project combining all skills T08–T14

You will need:

- E4 Educator v2.0 board with ESP32 fitted
- T12 and T13 completed — WiFi and LittleFS working
- min_spiffs.csv partition table already configured
- PC or phone browser on the same WiFi network as the E4

1 What is OTA?

OTA stands for Over-The-Air. It is the ability to update firmware on a deployed device without a USB cable, without physical access, and without interrupting the device any longer than the reboot takes. Every serious IoT device supports OTA — it is what separates a prototype from a deployable product.

For the E4 learner, OTA has an immediate practical benefit: once it is set up, you can update firmware from any browser on the same WiFi network. No more reaching around to find the USB-C port, no more disconnecting sensors during development.

1.1 How ESP32 OTA works

The ESP32 OTA mechanism uses two application partitions in flash — OTA_0 and OTA_1. The device boots from one while the other is written with the new firmware. On success, the boot partition pointer switches to the new firmware and the device reboots. If the new firmware fails to boot, the ESP32 can fall back to the previous version.

Step	What happens
1	Browser uploads new .bin firmware to E4 via HTTP POST
2	E4 writes incoming bytes to the inactive OTA partition
3	E4 verifies the written image (checksum)
4	E4 updates the boot partition pointer to the new image
5	E4 reboots and runs the new firmware
6	If the new firmware boots and calls esp_ota_mark_app_valid_cancel_rollback(), it is committed

1.2 Partition table requirement

⚠ OTA requires a specific partition table

The default Arduino partition table does not reserve OTA partitions. You must use a partition table that includes OTA_0 and OTA_1 — such as min_spiffs.csv or default.csv. If you use huge_app.csv or the default single-app table, OTA is not available. The E4 course uses min_spiffs.csv throughout Tier 2 precisely because it supports both OTA and LittleFS.

Verify your partition table is correct by checking platformio.ini:

```
; platformio.ini — OTA-compatible partition table
board_build.partitions  = min_spiffs.csv
board_build.filesystem  = littlefs

; min_spiffs.csv partition layout (4MB flash):
; nvs       0x9000   0x5000    (20KB  NVS)
; otadata   0xe000   0x2000    (8KB   OTA selection data)
; app0      0x10000  0x1E0000  (1.9MB OTA_0 - currently running)
; app1      0x1F0000 0x1E0000  (1.9MB OTA_1 - update target)
; spiffs    0x3D0000 0x30000   (~192KB filesystem)
```

2 ElegantOTA

ElegantOTA is a library that wraps the ESP32 OTA mechanism in a clean web interface. It hosts an upload page at /update on the existing web server, accepts a firmware .bin file, flashes it, and reboots. It integrates cleanly with ESPAsyncWebServer established in T13.

2.1 Add ElegantOTA to platformio.ini

```
lib_deps =
  bodmer/TFT_eSPI @ 2.5.43
  knolleary/PubSubClient @ 2.8
  adafruit/Adafruit AHTX0
  adafruit/Adafruit BusIO
  bblanchon/ArduinoJson @ 6.21.3
  https://github.com/me-no-dev/ESPAsyncWebServer
  https://github.com/me-no-dev/AsyncTCP
  ayushsharma82/ElegantOTA @ 2.2.9

; ElegantOTA requires ELEGANTOTA_USE_ASYNC_WEBSERVER
build_flags =
  ; ... all existing flags ...
  -DELEGANTOTA_USE_ASYNC_WEBSERVER=1
```

2.2 Minimal OTA integration

Adding OTA to any existing T13 web server project requires three changes — an include, a begin() call, and a loop() call:

```
#include <ElegantOTA.h>

// In setupWebServer() — add after server routes, before server.begin():
ElegantOTA.begin(&server);

// In loop() — add after mqtt.loop():
ElegantOTA.loop();

// That is all. Navigate to http://<E4-IP>/update in any browser.
// Select the firmware .bin file and click Update.
```

Key point

ElegantOTA.loop() must be called every loop() pass. Without it, the OTA process stalls mid-transfer. It is lightweight when no update is in progress — just a flag check. Treat it exactly like mqtt.loop() — mandatory, always called.

2.3 Building the firmware binary

PlatformIO builds the uploadable .bin file automatically during every build. To find it:

1. In VS Code, press Ctrl+Alt+B to build (or click the tick in the toolbar)
2. The .bin file appears at: .pio/build/e4_educator_v2/firmware.bin
3. Navigate to http://<E4-IP>/update in your browser

4. Click "Firmware" tab, choose the .bin file, click Update
5. The E4 flashes and reboots with the new firmware automatically

Finding the E4 IP address

The E4 prints its IP address to the Serial monitor at startup after WiFi connects. It also shows in the TFT footer when the dashboard is running. Note the IP — it changes on each boot unless you configure a DHCP reservation on your router for the E4's MAC address.

3 OTA progress on the TFT display

ElegantOTA provides callback hooks for start, progress, and end events. Use these to show a progress bar on the TFT so the learner can see the update happening.

```
#include <TFT_eSPI.h>
#include <ElegantOTA.h>

TFT_eSPI    tft;
TFT_eSprite spr = TFT_eSprite(&tft);

#define COL_BG      0x0000
#define COL_HEADER  0x0319
#define COL_TEXT    0xFFFF
#define COL_OK      0x07E0
#define COL_WARN    0xFFE0
#define COL_ACCENT  0xFD20

void otaShowProgress(int percent) {
  spr.fillSprite(COL_BG);

  spr.setTextColor(COL_ACCENT, COL_BG);
  spr.setTextSize(2);
  spr.setCursor(8, 20);
  spr.print("OTA Update");

  spr.setTextColor(COL_TEXT, COL_BG);
  spr.setTextSize(1);
  spr.setCursor(8, 50);
  spr.printf("Flashing firmware: %d%%", percent);

  // Progress bar
  int barW = map(percent, 0, 100, 0, 220);
  spr.drawRect(8, 70, 224, 20, COL_TEXT);
  if (barW > 0)
    spr.fillRect(9, 71, barW, 18, COL_OK);

  spr.setTextColor(COL_LABEL, COL_BG);
  spr.setTextSize(1);
  spr.setCursor(8, 110);
  spr.print("Do not power off during update.");

  spr.pushSprite(0, 80);
}

void setupOTA() {
  ElegantOTA.begin(&server);

  ElegantOTA.onStart([]() {
    Serial.println("OTA update started.");
    // Pause MQTT and sensors during update
    // Draw OTA screen chrome
    tft.fillScreen(COL_BG);
```

```
    tft.fillRect(0, 0, 240, 40, COL_HEADER);
    tft.setTextColor(COL_TEXT, COL_HEADER);
    tft.setTextSize(2);
    tft.setCursor(8, 12);
    tft.print("Updating firmware");
    otaShowProgress(0);
  });

  ElegantOTA.onProgress([](size_t current, size_t total) {
    int pct = (total > 0) ? (current * 100 / total) : 0;
    otaShowProgress(pct);
    Serial.printf("OTA progress: %d%%\n", pct);
  });

  ElegantOTA.onEnd([](bool success) {
    if (success) {
      Serial.println("OTA success. Rebooting...");
      spr.fillSprite(COL_BG);
      spr.setTextColor(COL_OK, COL_BG);
      spr.setTextSize(2);
      spr.setCursor(8, 40);
      spr.print("Update complete!");
      spr.setTextSize(1);
      spr.setCursor(8, 70);
      spr.print("Rebooting...");
      spr.pushSprite(0, 80);
      delay(2000);
    } else {
      Serial.println("OTA FAILED.");
      spr.fillSprite(COL_BG);
      spr.setTextColor(COL_ERR, COL_BG);
      spr.setTextSize(2);
      spr.setCursor(8, 40);
      spr.print("Update FAILED");
      spr.setCursor(8, 70);
      spr.print("Previous FW retained.");
      spr.pushSprite(0, 80);
    }
  });
}
```

4 Firmware versioning

Once OTA is in place, version tracking becomes essential. When multiple versions of firmware exist and devices update independently, you must be able to tell exactly what version any device is running.

4.1 Semantic versioning in build_flags

The Fundrum convention defines version and date as build_flags so they are always correct regardless of when or where the build is performed:

```
; platformio.ini — update before every release
build_flags =
  -DFW_VERSION=\"1.2.0\"
  -DFW_DATE=\"2026-05-15\"
  ; ... all other flags ...

; In code — always use these defines, never hardcoded strings
Serial.printf("Firmware v%s built %s\n", FW_VERSION, FW_DATE);
tft.printf("FW: %s  %s", FW_VERSION, FW_DATE);

; Version also published to MQTT on startup:
char fwMsg[32];
snprintf(fwMsg, sizeof(fwMsg), "%s/%s", FW_VERSION, FW_DATE);
mqtt.publish("e4/firmware", fwMsg, true);  // Retained
```

4.2 Semantic version numbering

Use three-part MAJOR.MINOR.PATCH versioning:

Part	When to increment	Example
MAJOR	Breaking change — existing behaviour removed or incompatible	1.x.x → 2.0.0
MINOR	New feature added, backward compatible	1.1.x → 1.2.0
PATCH	Bug fix, no new features	1.2.0 → 1.2.1

5 OTA safety practices

OTA is powerful and convenient, but an update that bricks a remote device is a serious problem. These practices keep updates safe.

5.1 The watchdog and rollback

The ESP32 has a hardware watchdog timer. If the new firmware crashes immediately after update, the watchdog triggers a reboot. Without rollback protection, the device reboots into the same broken firmware in an infinite loop.

ElegantOTA with the ESP-IDF OTA API handles the valid/invalid marking automatically — if the new firmware completes setup() and starts loop() successfully, it is marked valid. If it crashes before that, the bootloader falls back to the previous OTA partition on next boot.

```
// Best practice: call this early in loop() once the firmware
// has proven itself operational
#include <esp_ota_ops.h>

bool otaMarkedValid = false;

void loop() {
  // Mark firmware valid after first successful sensor read
  // This commits the new partition and disables rollback
  if (!otaMarkedValid) {
    esp_ota_mark_app_valid_cancel_rollback();
    otaMarkedValid = true;
    Serial.println("OTA: firmware marked valid.");
  }

  // ... rest of loop ...
}
```

5.2 Update-safe firmware design

Design firmware so that an OTA update at any moment leaves the device in a known state:

- Save settings to NVS before any long operation — they survive the reboot
- Close SD log files cleanly — the onStart() callback is the right place
- Publish MQTT "offline" LWT is handled by the broker automatically on disconnect
- Publish MQTT "updating" status in onStart() callback so Node-RED knows why the device went offline
- Never update OTA while a critical operation is in progress — use a flag to defer
- Always test new firmware on a bench unit before deploying to a remote location

5.3 Filesystem OTA

ElegantOTA also supports uploading a new LittleFS filesystem image. On the /update page, click the "Filesystem" tab instead of "Firmware", then upload the filesystem.bin file. This file is built by PlatformIO when you select Build Filesystem Image and is found at .pio/build/e4_educator_v2/littlefs.bin.

Filesystem OTA erases all LittleFS content

Uploading a new filesystem image completely erases the existing LittleFS partition and replaces it with the new image. Any runtime-generated files (logs, updated config.json) are lost. Design your config system so that missing files fall through to defaults safely — exactly as implemented in T13.

6 Complete project — production-ready E4 firmware

This is the Tier 2 capstone project. It combines every skill from T08 through T14 into a single firmware that is genuinely production-ready: TFT dashboard, touch UI, SD logging, NVS settings, WiFi, MQTT, LittleFS web server, smooth fonts, and OTA updates — all running simultaneously, all non-blocking.

6.1 platformio.ini

```
[env:e4_educator_v2]
platform   = espressif32@6.6.0
board      = esp32dev
framework  = arduino
monitor_speed  = 115200
monitor_port   = COM10
upload_port    = COM10

; OTA-compatible partition with LittleFS
board_build.partitions = min_spiffs.csv
board_build.filesystem = littlefs

lib_deps =
  bodmer/TFT_eSPI @ 2.5.43
  knolleary/PubSubClient @ 2.8
  adafruit/Adafruit AHTX0
  adafruit/Adafruit BusIO
  bblanchon/ArduinoJson @ 6.21.3
  https://github.com/me-no-dev/ESPAsyncWebServer
  https://github.com/me-no-dev/AsyncTCP
  ayushsharma82/ElegantOTA @ 2.2.9

build_flags =
  -DFW_VERSION=\"1.0.0\"
  -DFW_DATE=\"2026-05-15\"
  -DLED_RED=16 -DLED_GREEN=26 -DLED_BLUE=32
  -DBTN_NEXT=13 -DBTN_ENT=14
  -DTFT_CS=5 -DTFT_DC=17 -DTFT_BL=12
  -DTFT_MOSI=23 -DTFT_SCLK=18 -DTFT_MISO=19
  -DTOUCH_CS=33 -DSD_CS=15
  -DI2C_SDA=21 -DI2C_SCL=22
  -DONE_WIRE=4 -DLDR=39 -DMIC_ANALOG=36
  -DMEMS_BCLK=32 -DMEMS_WS=35 -DMEMS_SD=34
  -DDAC_AUDIO=25 -DPIEZO=27
  -DUSER_SETUP_LOADED=1 -DILI9341_DRIVER=1
  -DTFT_WIDTH=320 -DTFT_HEIGHT=240
  -DTFT_MISO=19 -DTFT_MOSI=23 -DTFT_SCLK=18
  -DTFT_CS=5 -DTFT_DC=17 -DTFT_RST=-1
  -DLOAD_GLCD=1 -DLOAD_FONT2=1 -DLOAD_FONT4=1
  -DLOAD_GFXFF=1 -DSMOOTH_FONT=1
  -DSPI_FREQUENCY=27000000
  -DSPI_READ_FREQUENCY=20000000
  -DSPI_TOUCH_FREQUENCY=2500000
  -DDEFAULT_SSID=\"lab_wifi\"
  -DDEFAULT_PASS=\"fundrum1\"
  -DDEFAULT_BROKER=\"192.168.1.79\"
```

```
  -DELEGANTOTA_USE_ASYNC_WEBSERVER=1
```

6.2 Feature map

T	Feature	Implementation
T08	TFT sprite dashboard	Three-zone layout, colour palette, sprite pipeline, chrome drawn once
T09	Touch UI	TouchZone footer buttons: READ, SETTINGS, OTA
T10	SD data logger	AHT20 + LDR CSV logging, log rotation, on/off toggle
T11	NVS settings	SettingsManager load/save/reset, brightness, alert thresholds
T12	WiFi + MQTT	Non-blocking state machine, PubSubClient, Node-RED integration, LWT
T13	LittleFS web server	Dashboard HTML, /api/sensors JSON, ArduinoJson config, smooth fonts
T14	OTA updates	ElegantOTA, TFT progress bar, firmware versioning, rollback safety

6.3 main.cpp structure

The capstone project is too large to reproduce in full here. The recommended structure separates concerns into focused functions. A clean loop() for this complexity level looks like:

```
void loop() {
  unsigned long now = millis();

  // ── Non-blocking subsystem updates ──────────────────────
  nextBtn.read();        // Must call every pass
  entBtn.read();         // Must call every pass
  wifiUpdate(now);       // WiFi state machine
  mqttUpdate(now);       // MQTT reconnect + loop
  ElegantOTA.loop();     // OTA receive + progress

  // ── Button events ──────────────────────────────────────
  Button::Event nextEv = nextBtn.lastEvent();
  Button::Event entEv  = entBtn.lastEvent();

  if (nextEv == Button::SHORT_PRESS) cyclePage();
  if (nextEv == Button::LONG_PRESS)  forceRead();
  if (entEv  == Button::SHORT_PRESS) openSettings();
  if (entEv  == Button::LONG_PRESS)  factoryResetCheck();

  // ── Touch events ───────────────────────────────────────
  uint16_t tx, ty;
  bool touched = tft.getTouch(&tx, &ty);
  handleTouch(tx, ty, touched, now);

  // ── Timed operations ───────────────────────────────────
  if (now - lastSensor >= settings.s.logInterval) {
```

```
    lastSensor = now;
    readSensors();
    logReading();
    publishSensors();
    updateDisplay();
  }

  if (now - lastFooter >= 1000) {
    lastFooter = now;
    updateFooter(now);
  }

  // ── OTA validity mark ──────────────────────────────
  if (!otaMarkedValid && now > 10000) {
    esp_ota_mark_app_valid_cancel_rollback();
    otaMarkedValid = true;
  }
}
```

The loop() reads as a clear list of responsibilities — no nested logic, no blocking waits, no magic numbers. Every subsystem call is a named function. This is what production embedded firmware looks like.

7 OTA workflow — step by step

The complete workflow for every firmware update after initial USB flash:

Step	Action	Notes
1	Make code changes in VS Code	Increment FW_VERSION in build_flags before release
2	Ctrl+Alt+B — build only	Generates .pio/build/.../firmware.bin
3	Note E4 IP from TFT footer or Serial	e.g. 192.168.1.105
4	Open browser: http://192.168.1.105/update	ElegantOTA upload page appears
5	Click Firmware tab, choose firmware.bin	Select from .pio/build/e4_educator_v2/
6	Click Update	TFT shows progress bar. Do not power off.
7	E4 reboots automatically	New FW_VERSION visible in TFT footer
8	If LittleFS changed: Filesystem tab, upload littlefs.bin	Only needed if data/ files changed

8 T14 summary and Tier 2 complete

Concept	Key takeaway
OTA mechanism	Dual OTA partitions. New firmware written to inactive partition. Boot pointer switches on success. Rollback if new firmware fails to start.
Partition requirement	OTA requires min_spiffs.csv or default.csv. huge_app.csv and single-app tables do not support OTA.
ElegantOTA	Three additions: #include, ElegantOTA.begin(&server), ElegantOTA.loop(). Hosts /update page automatically.
ELEGANTOTA flag	-DELEGANTOTA_USE_ASYNC_WEBSERVER=1 required in build_flags when using with ESPAsyncWebServer.
Progress callbacks	onStart(), onProgress(), onEnd() hooks drive TFT progress bar and Serial reporting during update.
Firmware versioning	FW_VERSION and FW_DATE in build_flags. Publish to MQTT retained topic on boot. Display in TFT footer always.
OTA validity mark	Call esp_ota_mark_app_valid_cancel_rollback() early in loop() after firmware proves operational. Commits new partition.
Filesystem OTA	Filesystem tab on /update page. Upload littlefs.bin. Erases all LittleFS content — design config to degrade gracefully.

★ Tier 2 complete ★

You have built a production-grade networked ESP32 device with a colour TFT dashboard, touch UI, SD logging, NVS settings, WiFi, MQTT, web server, smooth fonts, and OTA updates. Everything runs simultaneously and non-blocking. That is real embedded systems engineering.

Tier 3 — Integrated projects — applies all of this to complete real-world applications.

What's next — Tier 2 remaining

- T15 — ESP-NOW: peer-to-peer wireless communication between ESP32 devices without a router or broker. Perfect for sensor networks, remote controls, and multi-node projects.
- T16 — I2S audio and MEMS mic: the ICS43434 MEMS microphone already on the E4 board, I2S peripheral configuration, FFT analysis, and real-time spectrum display on the TFT.